DESIGN AND IMPLEMENTATION OF A WEB APPLICATION SECURITY RISK DETECTION AND REMEDIATION DEMONSTRATION SYSTEM BASED ON OWASP TOP 10

A Hybrid Approach Combining OWASP Juice Shop and Custom Flask Modules for Modern Web Vulnerability Mitigation

Course: 03CIT4067 Cybersecurity Project **Group No:** Group 7 (Topic 15)

Supervisor: Dr. LIAM XU **Date:** November 3, 2026

Team Members: WONG Lai Man (Leader), KHAN MUHAMMAD MUSTAFA, ZOUTAO

1. PROJECT TITLE

Design and Implementation of a Web Application Security Risk Detection and Remediation Demonstration System Based on OWASP Top 10: A Hybrid Approach Combining OWASP Juice Shop and Custom Flask Modules for Modern Web Vulnerability Mitigation.

2. IDENTIFIED ISSUE(S)

Web application security remains a critical challenge in the modern digital landscape. Specifically, this project addresses three concrete issues: **(1)** The persistence of Injection and Security Misconfiguration (A05) vulnerabilities which continue to dominate the threat landscape and allow unauthorized authentication bypass (OWASP Foundation, 2025); **(2)** The systemic operational flaw of Mishandling of Exceptional Conditions (A10) leading to "fail-open" states that inadvertently grant administrative privileges during unhandled runtime exceptions (GitLab, 2026); and **(3)** The lack of integrated, code-level remediation demonstrations in existing automated dynamic security testing tools (Zenodo, 2025a).

Despite the widespread availability of scanner tools like OWASP ZAP, existing software primarily focuses on vulnerability identification and logging without showing step-by-step secure code repairs. Developers struggle to bridge the gap between risk flags and implementing secure architectural controls that transition systems to a robust "fail-closed" architecture (Zenodo, 2025b).

3. PROJECT GOALS AND OBJECTIVES - SIGNIFICANCE OF STUDY

Project Goal (Vision): To design, deploy, and evaluate a hybrid web application security demonstration system that effectively detects, analyzes, and remediates critical web vulnerabilities framed within the OWASP Top 10: 2025 standard.

SMART Objectives:

- **Objective 1:** Successfully deploy the OWASP Juice Shop target environment via Docker containerization and build custom Python Flask modules demonstrating A10 (Fail-Open vs. Fail-Closed) exception handling by Week 9 of Term 1.
- **Objective 2:** Achieve a quantifiable reduction in security risk scores (eliminating High/Medium CSP and CORS alerts) as validated through OWASP ZAP automated vulnerability audits in Term 2.
- **Objective 3:** Produce a comprehensive technical report, STRIDE threat modeling analysis, and a 15-20 minute live demonstration showcasing code-level defensive programming.

Significance of Study: This research directly benefits junior software engineers and cybersecurity students by providing a visual, side-by-side comparative analysis of vulnerable and remediated backend code. Academically, it operationalizes the 2025 OWASP framework updates into a practical laboratory environment (McGraw, 2006).

4. PROJECT FEASIBILITY

4.1 Technical Feasibility: The team maintains strong technical competencies in Python, Flask middleware, Docker deployment, and web penetration testing (Stuttard & Pinto, 2011). Utilizing open-source targets guarantees technical viability without proprietary code constraints.

4.2 Data & Tools Feasibility: All tools (OWASP Juice Shop, OWASP ZAP, Kali Linux) are open-source and freely accessible. Synthetic vulnerability datasets eliminate privacy concerns.

4.3 Time & Economic Feasibility: Structured across two academic semesters, the timeline aligns with bi-weekly project milestones. The project incurs zero financial cost by leveraging local virtualization.

4.4 Risk Management Matrix

Risk Description	Likelihood	Mitigation Strategy
Network/hardware constraints preventing local Docker deployment for individual members.	Medium	Task Reallocation: Team Leader shifts local container execution to local members while delegating Red Team documentation and log analysis.
Automated scanner (ZAP) generates excessive false positives.	Low	Manual verification of findings; target scan rules specifically to A05 and A10 vulnerability classes.

5. PROPOSED SOLUTION(S)

5.1 System Architecture: As illustrated in **Figure 1**, the solution implements a hybrid architecture. OWASP Juice Shop handles standard web vulnerability testing, while a custom Flask module isolates A10 exception handling.

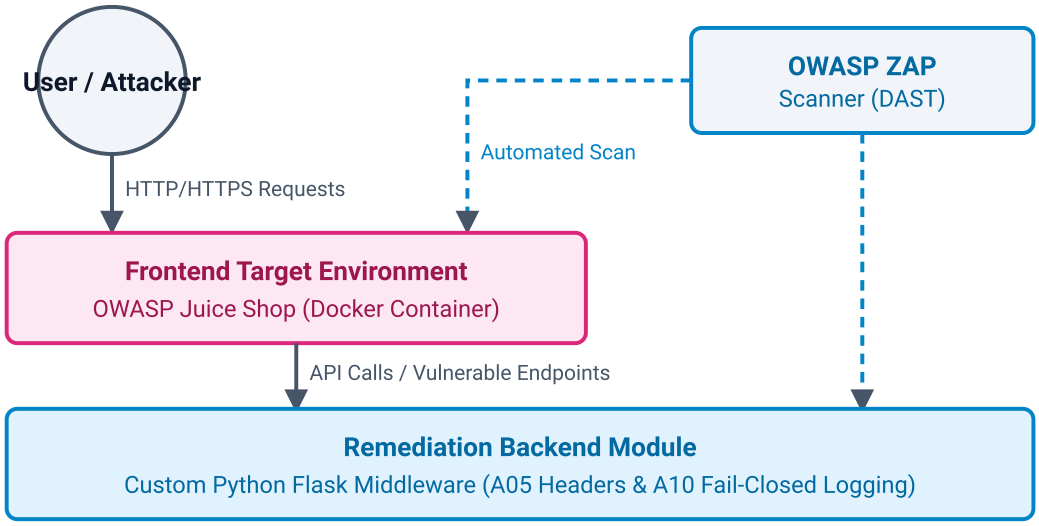


Figure 1: Hybrid System Architecture Diagram (Juice Shop Target + Custom Flask Remediation Module + ZAP DAST)

5.2 Main Functional Modules: (1) Vulnerability Target Module: Juice Shop container simulating real-world attack vectors. (2) Remediation Module: Flask backend enforcing HTTP security headers (A05) and fail-closed JSON audit logging (A10).

5.3 Technology Stack and Justification:

- **OWASP Juice Shop:** Selected as the primary target environment because it provides a comprehensive, modern, and intentionally vulnerable web application covering the majority of OWASP Top 10 categories out-of-the-box (Kimminich, 2024; OWASP Foundation, 2026).
- **Python Flask:** Chosen for the custom remediation module due to its lightweight nature and ease of integrating custom middleware for security headers (A05) and structured logging (A10).
- **OWASP ZAP:** Utilized as the automated dynamic application security testing (DAST) tool to provide quantitative pre- and post-remediation vulnerability metrics (Checkmarx, 2026).
- **Docker:** Used to containerize the Juice Shop instance, ensuring environment consistency and isolating the vulnerable application from the host system during testing.

5.4 Evaluation Method: Quantitative pre- and post-remediation ZAP scan vulnerability counts combined with qualitative Red Team manual verification.

6. ANTICIPATED ACCOMPLISHMENT

- **Proof of Concept System:** Fully functional Docker container suite demonstrating vulnerable vs. secured endpoints side-by-side.
- **Technical Documentation:** Comprehensive report including STRIDE threat models (OWASP Foundation, 2023) and code snippets.
- **Academic Deliverables:** Completed Working Journals, Presentation Deck, and a 15-20 minute live video demonstration.

7. SCHEDULE

The project follows an Agile Scrum framework broken into bi-weekly sprints across two terms, as detailed in **Figure 2**. Task tracking and version control are managed using **Jira** and **GitHub**, respectively.

- **Term 1 (Phase 1 & 2):** Proposal, System Design, and Implementation. *(Owner: All Members; Tool: Jira / GitHub)*
- **Term 2 (Phase 3):** Security Testing (ZAP), Report Integration, and Final Presentation. *(Owner: Team Leader; Tool: MS Word / ZAP)*

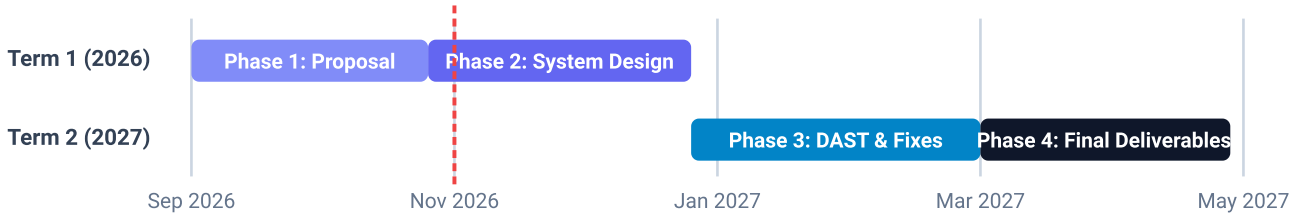


Figure 2: Two-Term Project Schedule Roadmap (Gantt Chart)

8. DIVISION OF LABOR

Member Name	Role	Specific Modules / Tasks Owned	Schedule Phases Covered	Workload (%)
WONG Lai Man	Team Leader	System Architecture, STRIDE Modeling, Proposal Integration, APA 7th Review, Gantt Schedule.	Phase 1, 2, 3, 4	100%
KHAN M. MUSTAFA	Developer / Tester	Docker Deployment, A05/A10 Code Remediation, OWASP ZAP Scans, Chapter 8 Lead.	Phase 2, 3	100%
ZOUTAO	Red Team Analyst	A01-A10 Attack Methodology, Chapter 5 Lead, Presentation Slides & Threat Diagrams.	Phase 1, 2, 3, 4	100%

Declaration: Division of labor agreed upon by all team members. WONG Lai Man acts as designated Group Leader.

9. REFERENCE LIST (APA 7TH FORMAT)

1. Checkmarx. (2026). *ZAP (Zed Attack Proxy) documentation*. https://www.zaproxy.org/docs/
2. GitLab. (2026, January 7). *OWASP Top 10 2025: What's changed and why it matters*. GitLab Blog. https://about.gitlab.com/blog/2025-owasp-top-10-whats-changed-and-why-it-matters/
3. Kimminich, B. (2024). *OWASP Juice Shop: The most trustworthy online store out there*. Packt Publishing.
4. McGraw, G. (2006). *Software security: Building security in*. Addison-Wesley Professional.
5. OWASP Foundation. (2023). *OWASP Threat Modeling Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html
6. OWASP Foundation. (2025). *OWASP Top 10: 2025: The ten most critical web application security risks*. https://owasp.org/www-project-top-ten/
7. OWASP Foundation. (2026). *OWASP Juice Shop documentation*. https://owasp.org/www-project-juice-shop/
8. Stuttard, D., & Pinto, M. (2011). *The web application hacker's handbook: Finding and exploiting security flaws* (2nd ed.). Wiley.
9. Zenodo. (2025a). *Mishandling of Exceptional Conditions Vulnerability: A Detailed Study*. https://zenodo.org/records/17867093
10. Zenodo. (2025b). *Software Supply Chain Failures: A detailed vulnerability study with applicable remediations for PHP-based applications*. https://zenodo.org/records/17866839